HW6: Scalability, Report

Part II: Finding the Bottleneck

The product search service stores 100,000 products in memory (sync.Map), checks exactly 100 per search for a fixed compute cost, and runs on a single Fargate task (0.25 vCPU, 512 MB). I load tested it with FastHttpUser against the deployed public IP and watched CloudWatch CPU/Memory next to my Homework 5 app-level signals.

Load	req/s	median	p95	p99	CPU peak	Memory
5 users (baseline)	36	86ms	92ms	160ms	~8%	12%
100 users (breaking)	657	96ms	150ms	240ms	74%	12%

What happened as load increased: throughput climbed to ~657 req/s and tail latency grew (p95 92ms to 150ms, p99 160ms to 240ms) while median held near baseline. CPU rose from ~8% idle to 74% sustained; memory stayed flat at 12% the entire time.

Optimize or scale: the evidence points to scaling, not code optimization. The search is already bounded at 100 checks, so the algorithm cannot be made cheaper. The bottleneck is CPU (74% under load), not memory (flat 12%) and not the code. Doubling CPU (256 to 512 units) would help because the work is CPU bound, and so would adding instances, which Part III demonstrates.

Dashboard vs CloudWatch: CloudWatch CPU told me the infrastructure was the constraint (CPU climbing toward saturation). My app-level dashboard told me what users felt, the rising p95/p99 tail. CPU alone would not have shown the latency degradation, and the latency alone would not have told me the cause was CPU. Both are needed: infra metrics to decide whether to scale, request metrics to confirm users are actually affected.

Part III: Horizontal Scaling with Auto Scaling

I deployed the same service (unchanged) behind an Application Load Balancer with auto scaling, via Terraform. Components: - ALB: single public entry point, distributes requests across healthy instances on port 8080. - Target Group: health checks /health every 30s, 2 successes to be healthy; registers Fargate tasks by IP. - Auto Scaling: target tracking on average CPU at 70%, min 2, max 4 instances.

Solving the Part II bottleneck: I reran the load that broke the single instance, then pushed to 300 users (3x the Part II breaking load).

Core test results: - 300 users, 639,830 requests, median 99ms, p95 220ms, 0 failures, ~1,800 req/s. - Auto scaling triggered: CPU crossed 70% at 18:11 (average rose to ~74% then 95%), and the service scaled from 2 to 3 tasks at

18:16. It settled at 3 because 3 instances brought average CPU back below 70%, so no further scale-out was needed. - The load that overwhelmed one instance in Part II was absorbed with zero errors once spread across instances.

Resilience test: during operation I stopped one running task. The service dropped to 2/3 running, ECS automatically launched a replacement, and recovered to 3/3 in about 50 seconds. Throughout the kill, the ALB returned 200 on every health check, so the service never went down. This shows individual instance failure does not bring down the system.

Vertical vs horizontal scaling tradeoffs: vertical (bigger instance) is simple but has a ceiling and a single point of failure. Horizontal (more instances behind an ALB) adds fault tolerance and near linear capacity, at the cost of more moving parts (load balancer, health checks, scaling policy) and the need for stateless instances.

CPU vs request-metric autoscaling: the policy here triggers on average CPU. Watching my duration/error signals during the scale-out, CPU and p95 rose together, so for this CPU bound workload CPU is a fine trigger. For a workload bounded by I/O or downstream latency rather than CPU, average request duration or requests-in-flight would catch the pressure sooner, because latency can climb while CPU stays moderate. For this service CPU is the right trigger; for a latency-bound service I would scale on p95 duration.

Stress testing done

Baseline (5 users), breaking point (100 users on single instance), and a 3x stress (300 users) against the auto-scaled cluster, plus a deliberate instance kill during load to test resilience. All runs captured with Locust stats, CloudWatch CPU/Memory, and live task-count polling.